

lexsync: A cross-platform pipeline for multidimensional lexical optimisation and hardware-timed experiment generation

Pablo Bernabeu

2026-06-10

Psycholinguistic stimulus preparation is fragmented across tools that match words, tools that present them and the separate R and Python ecosystems in which those tools live. We present ‘lexsync’, a research-grade toolkit, released as a ‘CRAN’-ready R package and a structurally identical ‘PyPI’-ready Python package, that unifies the whole path from corpus to laboratory. From a word-frequency corpus in any of dozens of languages ‘lexsync’ selects stimuli that are matched in parallel across several lexical dimensions (length, frequency, orthographic neighbourhood density and OLD20), counterbalances them, reports the realised match quality and then generates ready-to-run ‘PsychoPy’ and ‘OpenSesame’ experiment scripts whose hardware triggers for electroencephalography are time-locked to stimulus onset in both targets. The matching is deterministic, so the two engines select identical matched stimuli; we demonstrate this on five worked examples in English, Spanish and Mandarin Chinese, the last of which shows that the workflow extends to a logographic writing system. The generation step requires no presentation software and no hardware, which keeps the whole workflow reproducible. ‘lexsync’ generalises the FAIR artificial-grammar workflow of an existing longitudinal study and is distributed under open licences with a machine-readable corpus registry.

1 Introduction

The credibility of psycholinguistic and neurolinguistic research rests on carefully controlled stimuli and on precise timing. Both are laborious to achieve, and the tools that support them are scattered. A researcher who wishes to contrast, say, high- and low-frequency words while equating length and orthographic neighbourhood typically matches the items in one program, lays out the trial structure in another and writes the trigger code that aligns each word with the electroencephalography (EEG) recording by hand. Each hand-off is an opportunity for error and a barrier to reuse. The ‘FAIR’ guiding principles (Wilkinson et al., 2016) — that research objects be findable, accessible, interoperable and reusable — apply as much to this preparatory machinery as to the data it ultimately yields, yet stimulus pipelines are seldom shared in a form that others can run end to end.

Two families of tool address parts of the problem. The first matches or equates word stimuli on lexical properties. ‘LexOPS’ (Taylor et al., 2020) generates controlled word lists for arbitrary factorial designs, with both per-dimension tolerances and weighted multivariate Euclidean

matching; ‘LIBRA’ (Lintz et al., 2021) equates lists across conditions using a resampling algorithm; and ‘LASTU’ (Itkonen et al., 2024) provides searchable access to a language-specific lexicon. These tools are mature and widely used, but they end at the stimulus list: none generates the experiment that presents it, and most are tied to a single language and to one programming ecosystem. The second family builds and runs experiments. ‘PsychoPy’ (Peirce et al., 2019) and ‘OpenSesame’ (Mathôt et al., 2012) offer precise spatial and temporal control over stimulus presentation and support the hardware triggers that ERP research requires, but they take the stimulus set as given and do nothing to construct or to balance it.

The gap, then, is not a missing capability but a missing integration. No single, installable framework spans many-language corpus access, parallel multidimensional matching, counterbalancing and the automated generation of hardware-timed presentation scripts, and none does so across both the R and the Python ecosystems and both major open presentation platforms. A second, subtler gap concerns timing. In the artificial-grammar workflow that motivated this work (González Alonso et al., 2025), EEG markers were sent from an experiment item that followed the word, so that only the first marker was even approximately aligned with stimulus onset; the others were sequence-ordered. This is a common pattern and an avoidable source of jitter.

We present ‘lexsync’, a toolkit that fills these gaps; both presentation targets time-lock the marker to stimulus onset (see below). It is released as two independent but structurally identical packages — a ‘CRAN’-ready R package and a ‘PyPI’-ready Python package — that share module names, function names and behaviour. From a corpus ‘lexsync’ selects matched stimuli, counterbalances and documents them, then writes a ‘PsychoPy’ script in which each onset trigger is bound to the exact screen flip on which the word appears, using `win.callOnFlip`, together with a complete ‘OpenSesame’ experiment. Crucially the matching is deterministic, so the R and Python engines select identical matched stimuli. ‘lexsync’ generalises the materials workflow of González Alonso et al. (2025) into a reusable, FAIR-compliant tool.

2 The lexsync packages

2.1 Architecture and cross-language design

Each package is organised into the same modules: **querying** (load a lexicon and compute lexical dimensions), **matching** (the constraint engine), **counterbalancing**, **validation** (match-quality reporting), **scripting** (experiment generation), **logging** (the automated run log) and **corpora** (the registry). A single orchestrator, **run_pipeline**, drives them in an explicit loop over designs and languages, so that adding a language or a design is a matter of supplying a configuration file and a lexicon rather than writing code. The two packages are deliberately lean: the R package depends only on ‘readr’, ‘yaml’, ‘stringdist’ and ‘jsonlite’ beyond base R, and the Python package on ‘pandas’, ‘numpy’, ‘PyYAML’, ‘rapidfuzz’ and ‘scipy’.

2.2 A many-language corpus registry

Stimuli are only as good as the corpus behind them. ‘lexsync’ reads a machine-readable registry that records, for each supported corpus, its language, access URL, licence and citation. Two connectors are provided. The first exposes the curated SUBTLEX family and the ‘openlexicon’ collection — for example SUBTLEX-UK for British English (Heuven et al., 2014), SUBTLEX-ESP for Spanish (Cuetos et al., 2011) and SUBTLEX-US (Brysbaert & New, 2009) — each individually citable and redistributable under CC BY-SA terms. The second is the ‘wordfreq’

package (Speer, 2022), which yields frequencies for roughly forty languages through a single dependency. The demonstrations below build their English, Spanish and Mandarin Chinese lexica with ‘wordfreq’, which aggregates subtitle and web corpora including the SUBTLEX lists; the registry documents the curated corpora — among them SUBTLEX-CH for Chinese (Cai & Brysbaert, 2010) — as alternatives. Small example lexica are bundled inside each package for offline use and testing, while full corpora are fetched on demand into a user cache, keeping the installed packages within distribution size limits.

2.3 The multidimensional matching algorithm

For every word ‘lexsync’ records its length, its frequency and two orthographic-neighbourhood measures: Coltheart’s N , the number of words of the same length differing by a single letter (Coltheart et al., 1977), and OLD20, the mean Levenshtein distance to the twenty nearest neighbours (Yarkoni et al., 2008). The neighbourhood measures are computed transparently from the lexicon rather than taken on trust.

Matching proceeds in two deterministic stages. First, a tolerance pre-filter keeps only candidates that fall within $M \pm k \cdot SD$ of the anchor condition on each control dimension, generalising the windowed selection of the original workflow. Second, every remaining condition is matched to the anchor item by item: each dimension is standardised, the standardised Euclidean distance from each candidate to the target item is computed and the nearest unused candidate is assigned, with ties broken by a stable, locale-independent key. Because no random number generator is used and string comparisons are performed in byte order in both engines, the R and Python packages select an identical matched selection from identical input — a parity we verify below.

The per-anchor procedure fixes one condition and matches the others to it, which is ideal when the manipulated dimension is statistically independent of the controls. When the manipulation is intrinsically correlated with a control dimension — as orthographic neighbourhood density is with word length and frequency — fixing one condition first can leave a residual difference on the control, because the anchor items need not have close counterparts in the other condition. For such designs ‘lexsync’ offers a `joint` matching method, selected with a single line in the design configuration. Rather than privileging one condition, it scores every cross-condition pair on the standardised control dimensions and greedily retains the n cheapest disjoint pairs, so only items that have a close counterpart in the opposite condition are kept. The two conditions are thereby equated on the controls even when the manipulation is confounded with them, at the cost of drawing from a slightly larger candidate pool. The method is as deterministic and cross-engine identical as the per-anchor one (costs are rounded before the byte-order tie-break), and it is the method used for the neighbourhood-density demonstrations below.

2.4 Counterbalancing and reporting

Matched items are assigned to presentation lists and a reproducible trial order, and a participant table crosses any counterbalancing factors. ‘lexsync’ then writes a match-quality report — per-condition descriptive statistics for every dimension, standardised mean differences against the anchor and a two one-sided tests (TOST) equivalence test (Lakens, 2017) — and an automated, comprehensive run log recording each stage, its parameters, the seed, package and corpus provenance and a fingerprint of every file written. The report makes the realised control explicit, in the spirit of the original workflow’s balance checks, and the run log makes a run auditable.

2.5 Hardware time-locking

The ‘scripting’ module emits two presentation targets from the same matched stimuli. The ‘PsychoPy’ script binds each onset trigger to the stimulus flip:

```
win.callOnFlip(port.setData, target_trigger)    # written on the onset flip
for frame in range(WORD_DURATION_FRAMES):
    word_stim.draw()
    win.flip()
    if frame == RESET_AFTER_FRAMES:
        win.callOnFlip(port.setData, 0)
```

A mock port prints the codes when no parallel-port driver is present, so the script runs end to end with no hardware. The ‘OpenSesame’ export is a complete, plain-text `.osexp`, generated programmatically so that its tab-sensitive format is always valid and modelled on a proven-working experiment from González Alonso et al. (2025). In it the word is drawn and shown from an inline script, and the marker is sent immediately after `show()` returns, supporting both a parallel-port and a serial backend with a test-mode fallback. Because `show()` blocks until the display refresh, the marker is written right after the verified flip, so both targets are time-locked to stimulus onset: ‘PsychoPy’ binds the marker to the flip with `win.callOnFlip`, and ‘OpenSesame’ writes it immediately after the refresh-blocking `show()`, the platform’s recommended method. We recommend a photodiode check in either case. Notably, neither the matching nor the script generation imports ‘PsychoPy’ or any serial library: generation only writes text, and the presentation software is needed solely at run time.

2.6 Reproducibility and FAIR compliance

‘lexsync’ declares its dependencies in a `DESCRIPTION` file for R and pinned requirements for Python, sets seeds from a single configuration file, and carries a ‘`CITATION.cff`’, an MIT licence for the code and CC BY-SA terms for the bundled corpus derivatives with full attribution. The R package passes `R CMD check --as-cran`; the Python package builds a wheel and a source distribution that pass `twine check`. Both ship unit test suites, including a mock-engine test that confirms the generated ‘PsychoPy’ script writes its onset trigger on the word’s first flip and a structural validator for the generated `.osexp`.

3 Worked examples

We illustrate ‘lexsync’ with five demonstrations. Four are alphabetic: a high- versus low-frequency contrast and a dense- versus sparse-neighbourhood contrast, each in English and in Spanish, drawn from thirty-thousand-word lexica. The frequency contrast matches items on length, neighbourhood density and OLD20; the neighbourhood contrast matches on length and frequency. The fifth is a high- versus low-frequency contrast in Mandarin Chinese, included to show that the pipeline generalises to a logographic writing system (see *Generalisation to a logographic script*, below). Each demonstration was produced by both the R and the Python engine. All tables and figures in this section are computed at render time from the generated output.

Table 1: *Manipulated and Controlled Standardised Mean Differences by Demonstration. Note.* Each demonstration, its manipulated dimension and the absolute standardised mean difference (Cohen’s d) of the manipulated dimension and of the most-different controlled dimension. The manipulation is large in every case; the controlled dimensions remain small.

Demonstration	Manipulated dimension	$ d $ manipulated	largest $ d $ controlled
English: frequency contrast	Frequency (Zipf)	5.27	0.04
English: neighbourhood contrast	Neighbourhood N	3.11	0.00
Spanish: frequency contrast	Frequency (Zipf)	5.55	0.08
Spanish: neighbourhood contrast	OLD20	2.42	0.00
Chinese: frequency contrast	Frequency (Zipf)	6.00	0.11

Table 1 shows that the manipulated dimension is strongly separated in every demonstration. The control dimensions, by contrast, show standardised mean differences well below the 0.5-standard-deviation bound. Figure 1 plots the absolute standardised mean difference of each dimension against the anchor; the manipulated dimension stands far to the right of the 0.5-standard-deviation reference bound (dashed line) while the controlled dimensions cluster near zero.

The two one-sided tests (TOST) procedure that ‘lexsync’ reports gives a more conservative reading of the same control. Table 2 lists, for the controlled dimensions of each demonstration, the standardised mean difference, the equivalence-test p value and the verdict at the 0.5- SD bound.

Table 2: *Equivalence (TOST) Results for the Controlled Dimensions. Note.* For each demonstration’s controlled (matched) dimensions: the absolute standardised mean difference against the anchor, the equivalence-test p value at the 0.5- SD bound and the verdict at the .05 level. R engine.

Demonstration	Dimension	$ d $	TOST	
			p	Equivalent
Chinese: frequency contrast	Length	0.000	0.000	TRUE
Chinese: frequency contrast	Neighbourhood N	0.071	0.004	TRUE
Chinese: frequency contrast	OLD20	0.106	0.007	TRUE
English: frequency contrast	Length	0.027	0.002	TRUE
English: frequency contrast	Neighbourhood N	0.038	0.002	TRUE
English: frequency contrast	OLD20	0.009	0.001	TRUE
English: neighbourhood contrast	Length	0.000	0.001	TRUE
English: neighbourhood contrast	Frequency (Zipf)	0.000	0.001	TRUE
Spanish: frequency contrast	Length	0.049	0.002	TRUE
Spanish: frequency contrast	Neighbourhood N	0.077	0.004	TRUE
Spanish: frequency contrast	OLD20	0.006	0.001	TRUE
Spanish: neighbourhood contrast	Length	0.000	0.001	TRUE
Spanish: neighbourhood contrast	Frequency (Zipf)	0.000	0.001	TRUE

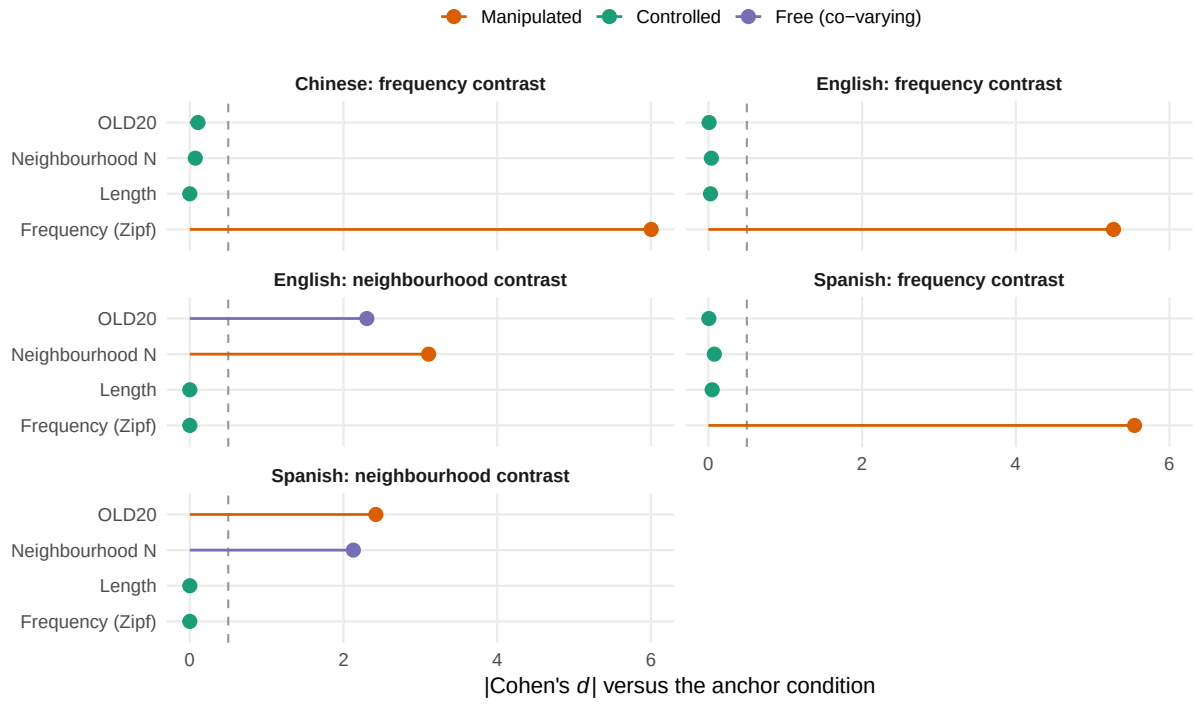


Figure 1: *Absolute Standardised Mean Differences by Dimension and Demonstration.* Note. Absolute standardised mean difference (Cohen's d) of each dimension against the anchor condition. The dashed line is the 0.5- SD reference bound (the smallest effect of interest for the equivalence test). Manipulated dimensions (orange) lie far to the right; matched dimensions (green) cluster near zero. In the neighbourhood contrasts OLD20 is free to co-vary (purple), reflecting its known correlation with Coltheart's N .

In every demonstration each controlled dimension is declared equivalent at the 0.5-*SD* bound ($n = 80$ per condition). The neighbourhood contrasts are the demanding case: neighbourhood density is intrinsically correlated with word length and frequency, so a per-anchor match leaves a residual on length (Cohen’s $d = 0.25$, just outside the bound). The `joint` matching method removes it. By selecting the best-matched dense–sparse pairs jointly it equates length and frequency almost exactly (Cohen’s $d = 0.00$ on both, well inside the bound) while the density contrast remains an order of magnitude larger ($d > 2$). The confound is thus a solved engineering problem rather than an inherent limit on the design, and ‘lexsync’ reports the realised equivalence transparently. We read the TOST as a conservative complement to the standardised mean difference (Lakens, 2017).

The per-condition descriptive statistics make the control concrete. In the English frequency contrast (Table 3) frequency is separated by roughly one and a half Zipf units while length, neighbourhood density and OLD20 are almost identical across conditions. In the English neighbourhood contrast (Table 4) neighbourhood density is separated by an order of magnitude with length and frequency held constant; OLD20, which is mathematically related to N , moves with the manipulation rather than being held constant, as expected.

Table 3: *Descriptive Statistics per Condition: English Frequency Contrast. Note.* R engine.

Condition	Dimension	n	Mean	SD	Min	Median	Max
high frequency	Length	80	4.700	1.427	3.00	4.000	8.00
high frequency	Frequency (Zipf)	80	5.578	0.369	5.20	5.455	6.98
high frequency	Neighbourhood N	80	7.812	8.344	0.00	5.000	34.00
high frequency	OLD20	80	1.629	0.541	1.00	1.600	3.10
low frequency	Length	80	4.662	1.350	3.00	4.000	7.00
low frequency	Frequency (Zipf)	80	4.064	0.170	3.82	4.035	4.40
low frequency	Neighbourhood N	80	7.513	7.480	0.00	5.000	24.00
low frequency	OLD20	80	1.624	0.518	1.00	1.600	2.70

Table 4: *Descriptive Statistics per Condition: English Neighbourhood-Density Contrast. Note.* R engine.

Condition	Dimension	n	Mean	SD	Min	Median	Max
dense neighbourhood	Length	80	4.787	0.650	4.00	5.000	6.00
dense neighbourhood	Frequency (Zipf)	80	4.249	0.311	3.81	4.225	5.15
dense neighbourhood	Neighbourhood N	80	8.363	3.530	5.00	7.500	20.00
dense neighbourhood	OLD20	80	1.448	0.197	1.00	1.500	1.75
sparse neighbourhood	Length	80	4.787	0.650	4.00	5.000	6.00
sparse neighbourhood	Frequency (Zipf)	80	4.249	0.311	3.81	4.225	5.15
sparse neighbourhood	Neighbourhood N	80	0.525	0.503	0.00	1.000	1.00
sparse neighbourhood	OLD20	80	1.970	0.253	1.60	1.900	2.95

3.1 Generalisation to a logographic script

The orthographic-neighbourhood measures operate on character strings, so the pipeline is not tied to alphabetic writing. To show this concretely, the fifth demonstration is a high- versus low-frequency contrast in Mandarin Chinese. Its lexicon is derived from the same engine as the others — `wordfreq`’s Chinese data (Speer, 2022) — and the two conditions are two-character

words, the dominant word form in Chinese. Here ‘length’ is the number of characters, and Coltheart’s N and OLD20 are computed over characters rather than letters; we refer to them as character-level neighbourhood measures. Adding the demonstration required no change to the matching, counterbalancing or scripting code: a design configuration names the Chinese lexicon and a glyph-complete font, and the existing machinery does the rest.

Table 5 shows the result. Frequency is separated by about one and a half Zipf units (Cohen’s $d = 6$), while character-length is held constant at two and the character-level neighbourhood measures are equated across conditions (both TOST-equivalent at the $0.5\text{-}SD$ bound). Chinese two-character words have dense orthographic neighbourhoods — tens of single-character substitutions each, an order of magnitude more than the alphabetic words above — and ‘lexsync’ equates this dense, skewed distribution as readily as the sparse alphabetic one. The generated ‘PsychoPy’ and ‘OpenSesame’ experiments set a CJK font, so the characters render directly; the selection is byte-identical across the R and Python engines, exactly as for the alphabetic demonstrations.

Table 5: *Descriptive Statistics per Condition: Mandarin Chinese Frequency Contrast.* *Note.* Length is in characters; neighbourhood N and OLD20 are character-level measures. R engine.

Condition	Dimension	n	Mean	SD	Min	Median	Max
high frequency	Length	80	2.000	0.000	2.0	2.000	2.00
high frequency	Frequency (Zipf)	80	5.324	0.294	5.0	5.245	6.56
high frequency	Neighbourhood N	80	69.825	64.807	8.0	51.500	307.00
high frequency	OLD20	80	1.038	0.111	1.0	1.000	1.55
low frequency	Length	80	2.000	0.000	2.0	2.000	2.00
low frequency	Frequency (Zipf)	80	3.810	0.201	3.5	3.810	4.20
low frequency	Neighbourhood N	80	65.662	52.159	14.0	51.500	199.00
low frequency	OLD20	80	1.028	0.074	1.0	1.000	1.25

Finally, because the matcher is deterministic the two engines select the same matched (word, condition) pairs rather than merely similar ones (the seeded trial order differs, as the R and Python random-number generators do). Across the five demonstrations all 800 of the selected (word, condition) pairs were identical between the R and the Python engine, and every statistic they report agrees to the precision shown. Table 6 makes this concrete for the English frequency contrast: the standardised mean difference of each dimension, computed independently in the two languages, coincides to the precision shown.

Table 6: *Cross-Engine Parity for the English Frequency Contrast.* *Note.* Each dimension’s standardised mean difference (Cohen’s d) against the anchor, computed independently by the R and Python engines. The columns are identical to three decimal places.

Dimension	Cohen’s d (R engine)	Cohen’s d (Python engine)
Length	0.027	0.027
Frequency (Zipf)	5.272	5.272
Neighbourhood N	0.038	0.038
OLD20	0.009	0.009

4 Discussion

‘lexsync’ unifies steps that are usually performed in separate tools and separate languages. Its contribution is integration rather than a single new capability: multidimensional matching exists in ‘LexOPS’ and ‘LIBRA’, and precise presentation exists in ‘PsychoPy’ and ‘OpenSesame’, but the path from a multilingual corpus to a hardware-timed, cross-platform experiment has not before been available as a single installable, dual-language tool. Two features are worth emphasising. First, the deterministic matcher selects identical matched (word, condition) sets in the R and Python engines, with the reported match statistics agreeing to the precision shown; only the seeded trial order, which depends on each language’s random-number generator, differs. A laboratory can therefore adopt ‘lexsync’ in whichever ecosystem it prefers without fear of divergent stimulus selection. Second, both presentation targets upgrade the sequence-ordered triggers of the original workflow to onset-aligned markers — flip-locked in ‘PsychoPy’, and written immediately after the refresh-blocking `show()` in ‘OpenSesame’ — a concrete methodological improvement for ERP research.

Some limitations remain. The Chinese demonstration shows that the pipeline generalises to a logographic script — corpus access, dimension derivation, matching, counterbalancing and CJK-font script generation all work unchanged — but the neighbourhood measures it equates there (Coltheart’s N and OLD20) are computed at the character level. They do not capture the sub-character structure that also shapes Chinese reading, such as radical, stroke-count and phonetic-radical information. Such measures are not intrinsic to the method; they enter as additional script-specific dimensions through the same extensible dimension system, and supplying them is left to future work. A photodiode check remains advisable for any onset-critical study, as for any software trigger. Future work includes additional presentation targets, a graphical interface and richer lexical dimensions such as concreteness and age of acquisition, each of which slots into the existing module structure.

5 Availability and open practices

‘lexsync’ is available as an R package and a Python package, with a shared corpus registry and the reproducible source of this manuscript. The code is released under the MIT licence and the bundled corpus derivatives under CC BY-SA 4.0, with full attribution and retrieval dates recorded alongside the data. The repository URL in the package metadata is a placeholder pending public release.

5.1 Extending lexsync

In keeping with the design, extensions have well-defined insertion points. To add a lexical dimension, edit `compute`-style functions in `querying` (`add_neighbourhood` and the dimension table in `config/schema.yaml`) in both packages. To add a presentation target, add an `export_<target>()` function in `scripting`. To add a corpus or a language, add an entry to `corpora/registry.yaml`; no code change is required for SUBTLEX-family or ‘wordfreq’ sources.

6 References

- Brysbaert, M., & New, B. (2009). Moving beyond Kučera and Francis: A critical evaluation of current word frequency norms and the introduction of a new and improved word frequency measure for American English. *Behavior Research Methods*, 41(4), 977–990. <https://doi.org/10.3758/BRM.41.4.977>
- Cai, Q., & Brysbaert, M. (2010). SUBTLEX-CH: Chinese word and character frequencies based on film subtitles. *PLoS ONE*, 5(6), e10729. <https://doi.org/10.1371/journal.pone.0010729>
- Coltheart, M., Davelaar, E., Jonasson, J. T., & Besner, D. (1977). Access to the internal lexicon. In S. Dornic (Ed.), *Attention and performance VI* (pp. 535–555). Erlbaum.
- Cuetos, F., González-Nosti, M., Barbón, A., & Brysbaert, M. (2011). SUBTLEX-ESP: Spanish word frequencies based on film subtitles. *Psicologica*, 32(2), 133–143.
- González Alonso, J., Bernabeu, P., Silva, G., DeLuca, V., Poch, C., Ivanova, I., & Rothman, J. (2025). Starting from the very beginning: Unraveling third language (L3) development with longitudinal data from artificial language learning and EEG. *International Journal of Multilingualism*, 22(1), 119–142. <https://doi.org/10.1080/14790718.2024.2415993>
- Heuven, W. J. B. van, Mandera, P., Keuleers, E., & Brysbaert, M. (2014). SUBTLEX-UK: A new and improved word frequency database for British English. *Quarterly Journal of Experimental Psychology*, 67(6), 1176–1190. <https://doi.org/10.1080/17470218.2013.850521>
- Itkonen, S., Häikiö, T., Vainio, S., & Lehtonen, M. (2024). LASTU: A psycholinguistic search tool for Finnish lexical stimuli. *Behavior Research Methods*, 56(6), 6165–6178. <https://doi.org/10.3758/s13428-024-02347-x>
- Lakens, D. (2017). Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4), 355–362. <https://doi.org/10.1177/1948550617697177>
- Lintz, E. N., Lim, P. C., & Johnson, M. R. (2021). A new tool for equating lexical stimuli across experimental conditions. *MethodsX*, 8, 101545. <https://doi.org/10.1016/j.mex.2021.101545>
- Mathôt, S., Schreij, D., & Theeuwes, J. (2012). OpenSesame: An open-source, graphical experiment builder for the social sciences. *Behavior Research Methods*, 44(2), 314–324. <https://doi.org/10.3758/s13428-011-0168-7>
- Peirce, J., Gray, J. R., Simpson, S., MacAskill, M., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, 51(1), 195–203. <https://doi.org/10.3758/s13428-018-01193-y>
- Speer, R. (2022). *rspeer/wordfreq: v3.0*. Zenodo. <https://doi.org/10.5281/zenodo.7199437>
- Taylor, J. E., Beith, A., & Sereno, S. C. (2020). LexOPS: An R package and user interface for the controlled generation of word stimuli. *Behavior Research Methods*, 52(6), 2372–2382. <https://doi.org/10.3758/s13428-020-01389-1>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., et al. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Yarkoni, T., Balota, D., & Yap, M. (2008). Moving beyond Coltheart’s N: A new measure of orthographic similarity. *Psychonomic Bulletin & Review*, 15(5), 971–979. <https://doi.org/10.3758/pbr.15.5.971>